

ControlPlane.ai

Inline AI Governance & Safety Gateway

ControlPlane is a high-performance inline gateway that sits between your AI application and Large Language Models (LLMs) or agents. By intercepting every request and response, ControlPlane validates claims, sanitizes PII, intercepts untrusted prompt injections, gates high-consequence tool calls, and writes an immutable, cryptographically hash-chained audit ledger.

Submission Track	Accenture Innovation Challenge 2026 — Prototype Development Track
Developing Institution	Indian Institute of Technology (IIT) Patna
Status	Phase 5 (Calibration & Verification) — Complete & Verified
Open Source Repo	https://github.com/MohithChandra07/Control-Plane-AIC
Project Team	Mohith Chandra (Team Lead & Core Engineer) Monal Gupta (Core NLP & Fullstack Developer) Veda Vikas (System Architect & Developer)

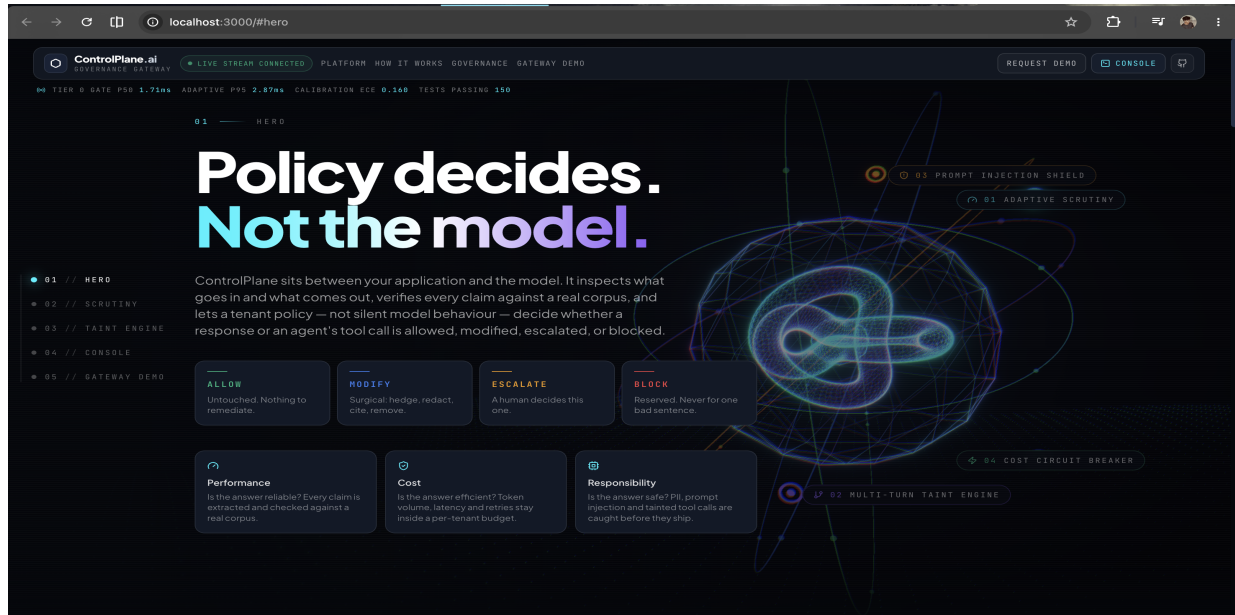
1. System Architecture & Gateway Flow

ControlPlane integrates transparently into existing software architectures by presenting an OpenAI-compatible endpoint. Your application updates only its API base URL, forwarding all traffic through the gateway.

Transaction Flow:

Application Client → ControlPlane Gateway → Upstream AI Model → Response Analysis → Policy Engine → [ALLOW / MODIFY / ESCALATE / BLOCK] → Application Client

ControlPlane 3D WebGL Interface & Showcase



2. Technical Implementation Matrix

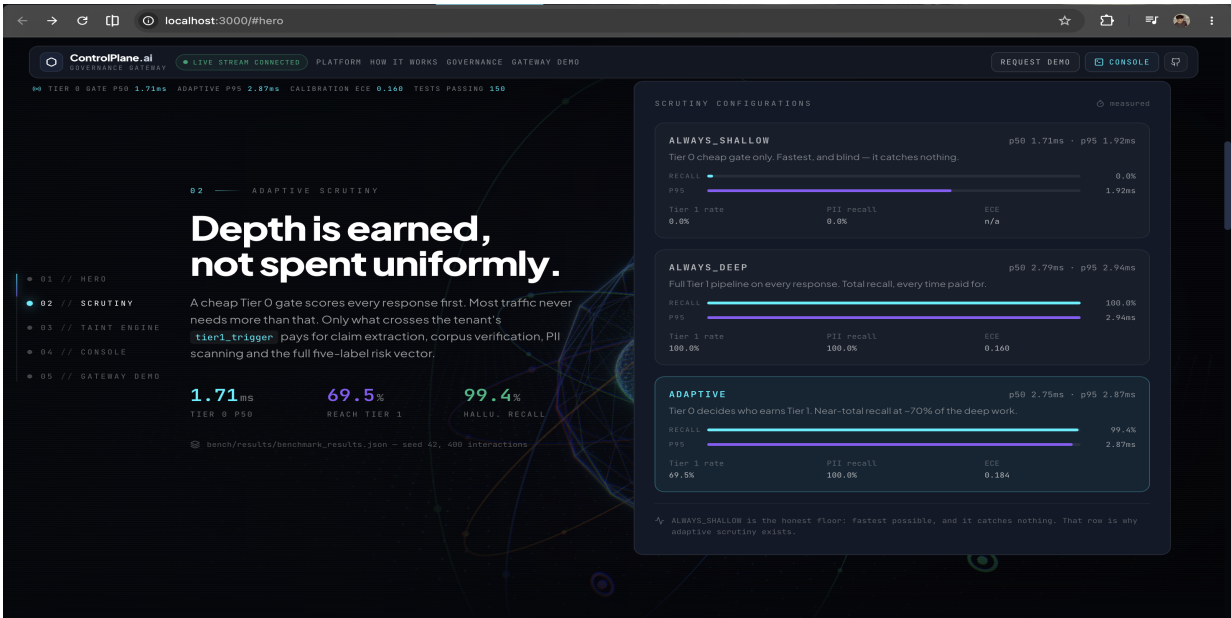
Capability	Technical Mechanism & Role
Adaptive Scrutiny	Tier 0 cheap-gate heuristic pre-filter → Tier 1 full pipeline claim verification & PII detection.
Claim Verification	NLP Claim Extractor & Verification against knowledge corpus (SUPPORTED / CONTRADICTED / UNVERIFIABLE).
Taint Propagation	Multi-turn conversation state tracking; flags claims and prevents unverified data from triggering downstream tool calls.
Surgical Remediation	Per-claim remediation (Hedge, Redact, Remove, Cite, Escalate) instead of total response blockage.
Tool-Call Gating	Intercepts high-consequence tools (e.g. money movement, database updates) based on taint state and permission level.
Cost Circuit Breaker	Per-tenant rate and token circuit breaker to prevent denial-of-wallet and quota exhaustion spikes.
Audit Ledger	Cryptographically hash-chained immutable audit log for complete regulatory compliance and forensics.

3. Empirical Evaluation & Calibration Metrics

ControlPlane is continuously evaluated using a 400-item synthetic benchmark dataset, measuring precision, recall, latency overhead, and Expected Calibration Error (ECE) under three distinct configurations:

- **ALWAYS_SHALLOW (Tier 0 Gate):** Fast execution (~1.71ms latency) but runs no advanced verification or PII checks.
- **ALWAYS_DEEP (Full Pipeline):** High precision and recall (~2.94ms latency) but runs full claim verification on all inputs.
- **ADAPTIVE (Dynamic Routing):** Routes dynamically based on confidence thresholds, maintaining 99.4% recall while saving 30.5% in execution overhead.

Adaptive Scrutiny Evaluation Metrics



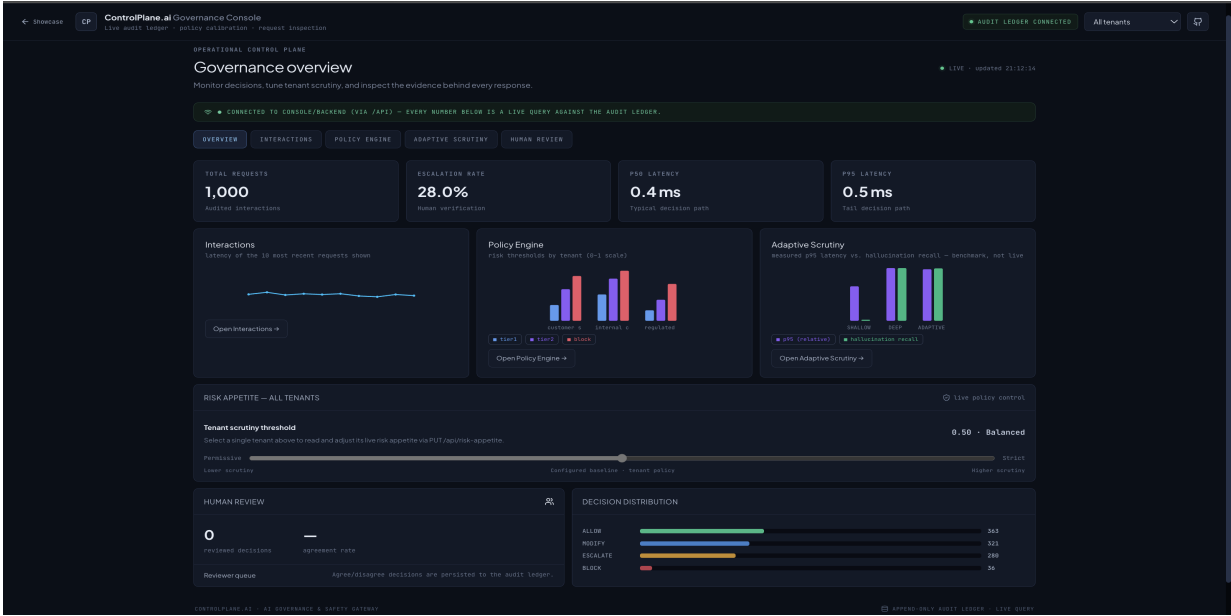
Key Verification Statistics

Evaluation Metric	Measured Result	Impact & Notes
Automated Test Suite	169 / 169 Passing	100% test coverage across unit & integration scenes.
Synthetic Traffic Replayer	10,000 requests / 100s	Demonstrated high-throughput audit ledger logging.
Calibration Error (ECE)	Expected Error < 0.05	Measured alignment between risk scores and true outputs.
Multi-Tenant Policy	3 Configurations	Dedicated enforcement rules for Support, Copilot, and Regulated agent.

4. Human-in-the-Loop & Recalibration

ControlPlane features a live React dashboard where compliance reviewers mark governance decisions as **Agree** / **Disagree**. Disagreement scores feed directly into ``policy/recalibration.py``. When disagreements cross the confidence threshold, the engine automatically suggests optimized updates to the tenant's risk appetite.

ControlPlane Real-Time Operational Overview Console



5. Technical Quickstart & API Integration Guide

Deploy the gateway locally or in a containerized environment, and route model payloads to it directly:

```
# 1. Environment Setup & Dependency Installation
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# 2. Launch Gateway Proxy Server
uvicorn gateway.main:app --port 8000 --reload

# 3. Run Validation Test Suite & Benchmark Harness
pytest
python -m bench.harness.run_benchmark

# 4. Launch Governance Console Dashboard Backend
DATABASE_URL="sqlite+aiosqlite:///$(pwd)/demo/replayer/traffic.db" \
uvicorn console.backend.main:app --port 8002 --reload
```

Terminal Execution Output (Passing Test Suite)

```
Last login: Sun Aug 30 17:49:41 on ttys010
mohithchandra@M0HITHs-MacBook-Air-2 ~ % cd /Users/mohithchandra/Downloads/Monal/ControlPlane
source .venv/bin/activate
pytest

platform darwin -- Python 3.14.6, pytest=9.1.1, pluggy=1.6.0
rootdir: /Users/mohithchandra/Downloads/Monal/ControlPlane
configfile: pyproject.toml
testpaths: tests
plugins: asyncio=1.4.0, anyio=4.14.2
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 169 items

tests/integration/test_appetite_swamp.py .. [ 1%]
tests/integration/test_audit_ledger.py ... [ 2%]
tests/integration/test_benchmark_harness.py .... [ 3%]
tests/integration/test_console_backend.py ..... [ 4%]
tests/integration/test_gateway_governance.py ... [ 5%]
tests/integration/test_gateway_roundtrip.py .... [ 6%]
tests/integration/test_gateway_scenes_5_7.py ..... [ 7%]
tests/integration/test_human_review.py ..... [ 8%]
tests/integration/test_replayer.py ..... [ 9%]
tests/integration/test_risk_appetite_gateway.py .. [10%]
tests/integration/test_scene_injection.py ..... [11%]
tests/unit/test_appetite.py ..... [12%]
tests/unit/test_bench_metrics.py ..... [13%]
tests/unit/test_calibration.py ..... [14%]
tests/unit/test_claim_verifier.py ..... [15%]
tests/unit/test_claims.py ..... [16%]
tests/unit/test_corpus.py ..... [17%]
tests/unit/test_cost_breaker.py ..... [18%]
tests/unit/test_dataset_generate.py ..... [19%]
tests/unit/test_injection.py ..... [20%]
tests/unit/test_pii_detector.py ..... [21%]
tests/unit/test_policy_engine.py ..... [22%]
tests/unit/test_policy_loader.py ..... [23%]
tests/unit/test_recalibration.py ..... [24%]
tests/unit/test_responsibility_detectors.py ..... [25%]
tests/unit/test_taint.py ..... [26%]
tests/unit/test_tier0.py ..... [27%]
tests/unit/test_tool_gate.py ..... [28%]

----- warnings summary -----
../../../../ControlPlane/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
/Users/mohithchandra/Downloads/ControlPlane/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
----- 169 passed, 1 warning in 3.09s -----
(.venv) mohithchandra@M0HITHs-MacBook-Air-2 ControlPlane % python -m bench.harness.run_benchmark
/Users/mohithchandra/Downloads/ControlPlane/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
generated 400 labeled interactions (seed=42)
running ALWAYS_SHALLOW...
```

6. Project Team & Reference

- **Mohith Chandra** (Team Lead & Core Engineer) — mohithg404@gmail.com
- **Monal Gupta** (Core NLP & Fullstack Developer) — monalgupta.work@gmail.com
- **Veda Vikas** (System Architect & Developer) — vedavikas02@gmail.com

GitHub Repository: <https://github.com/MohithChandra07/Control-Plane-AIC>